- It allows for the specification of rules for operations on the blockchain.
- It facilitates the implementation of policies for the transfer of assets in a decentralized network.
- It embeds functions that can be invoked by messages or function calls from participant accounts or other smart contract accounts. These messages and their input parameters, along with additional metadata such as the sender’s address and timestamp, result in transactions recorded in the distributed ledger of the blockchain.
- It acts as the software-based intermediator for decentralized blockchain-based applications.
- It adds programmability and intelligence to the blockchain through the specification of the parameters of its functions.

With all these crucial capabilities, a smart contract is indeed a core component of the decentralized blockchain application.

2.2 Design of a smart contract

Let’s begin our exploration of smart contract design with a simple example that will take you through the entire process, from problem statement to code deployment. In this first example, you’ll design a decentralized counter. Counters are common elements in everyday applications. Table 2.1 lists types of systems that use a counter. Turnstiles count the number of people entering and exiting an amusement park. A stock market index goes up or down based on the sales